



Security Audit

Celo 2nd (DeFi)

Table of Contents

Executive Summary	4
Project Context	4
Audit Scope	7
Security Rating	10
Intended Smart Contract Functions	11
Code Quality	17
Audit Resources	17
Dependencies	17
Severity Definitions	18
Status Definitions	19
Audit Findings	20
Centralisation	31
Conclusion	32
Our Methodology	33
Disclaimers	35
About Hashlock	36

CAUTION

THIS DOCUMENT IS A SECURITY AUDIT REPORT AND MAY CONTAIN CONFIDENTIAL INFORMATION. THIS INCLUDES IDENTIFIED VULNERABILITIES AND MALICIOUS CODE WHICH COULD BE USED TO COMPROMISE THE PROJECT. THIS DOCUMENT SHOULD ONLY BE FOR INTERNAL USE UNTIL ISSUES ARE RESOLVED. ONCE VULNERABILITIES ARE REMEDIATED, THIS REPORT CAN BE MADE PUBLIC. THE CONTENT OF THIS REPORT IS OWNED BY HASHLOCK PTY LTD FOR USE OF THE CLIENT.

Executive Summary

The Celo team partnered with Hashlock to conduct a security audit of their smart contracts. Hashlock manually and proactively reviewed the code in order to ensure the project's team and community that the deployed contracts are secure.

Project Context

The Celo project is a purpose-driven, Ethereum-anchored Layer-2 blockchain platform designed to enable fast, low-cost, and carbon-aware payments and decentralized finance (DeFi) applications globally. It supports gas payments in ERC-20 tokens and incorporates modular technologies, including an OP-Stack L2, EigenDA for scalable data availability, and a zkEVM (via Succinct SP1) for verified execution, together enabling one-second block times and sub-cent fees. By prioritizing accessibility, interoperability with Ethereum, and sustainability (for example, allocating 20 % of transaction fees to carbon offsets), Celo positions itself as an infrastructure foundation for real-world financial inclusion and on-chain economic growth.

Project Name: Celo

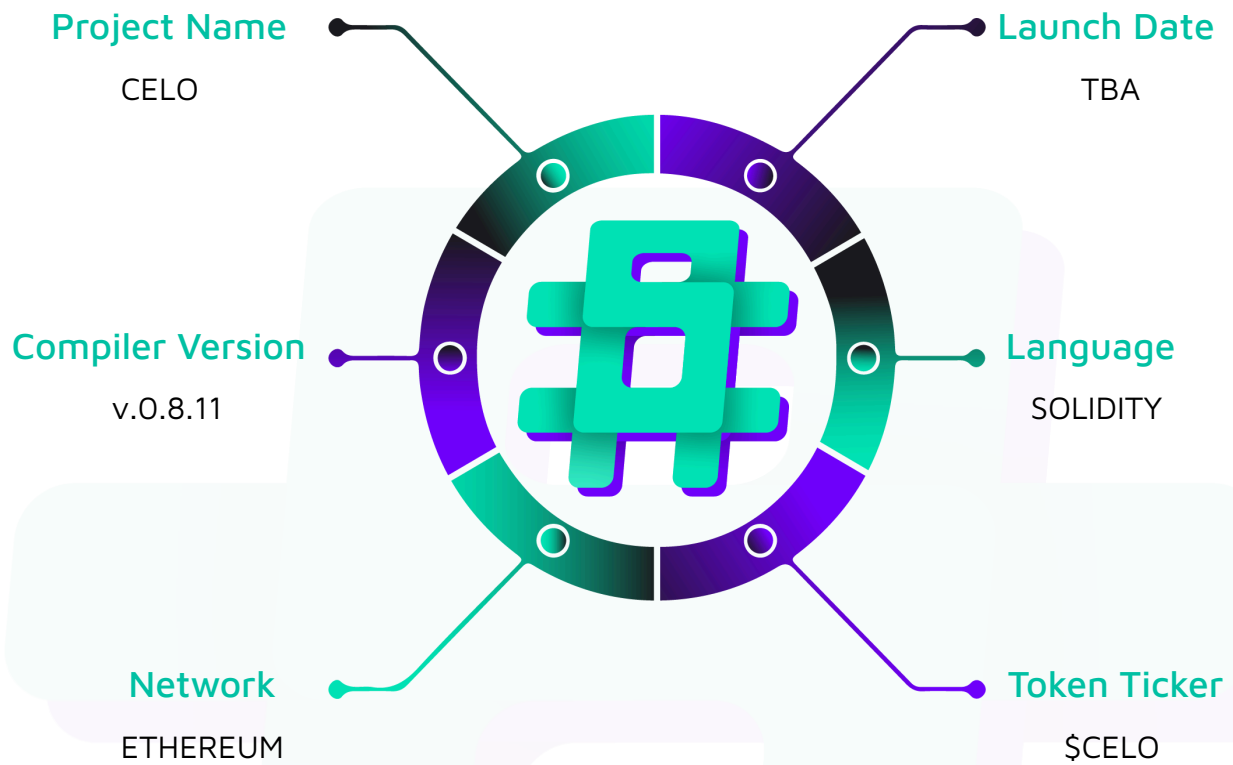
Project Type: DeFi

Compiler Version: 0.8.11

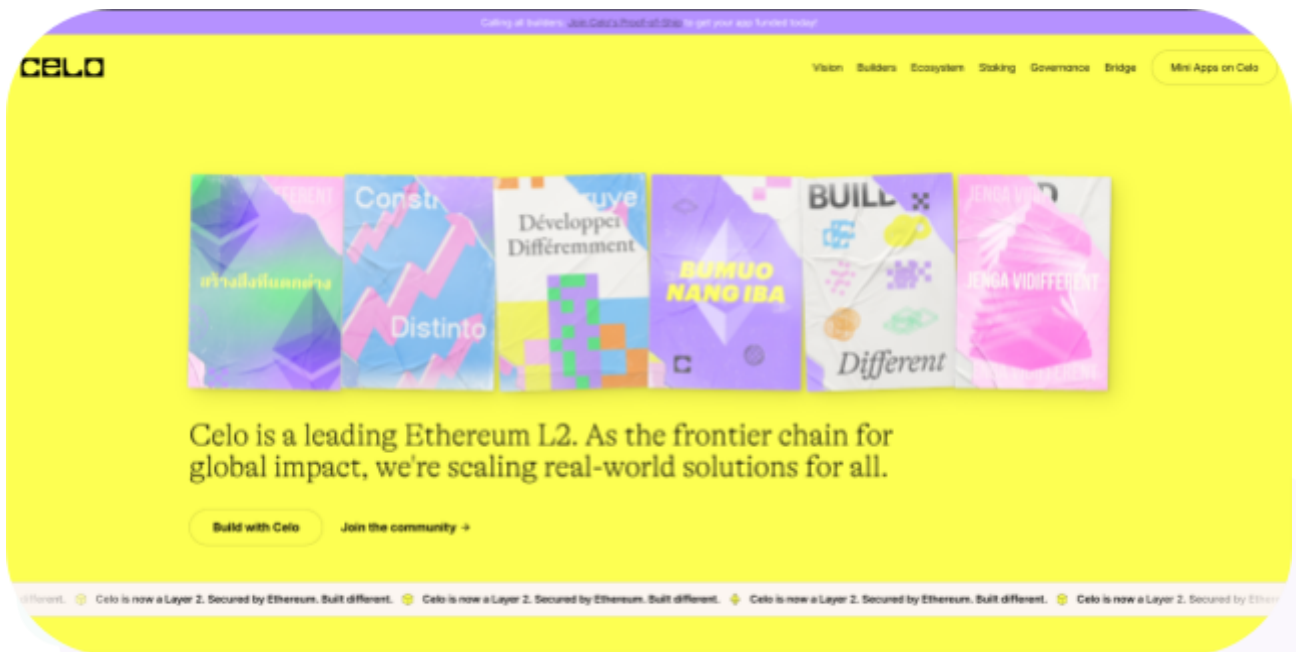
Website: <https://celo.org/>

Logo:



Visualised Context:

Project Visuals:



Discover Validators

Total Staked CELO: 106,377,900.5 CELO

Elected Minimum Votes: 562,838.9 CELO

Total Groups: 65 | Total Validators: 137

Search by name or address

All Eligible (65) | Elected (12) | Unelected (11) | Ineligible (2)

#	Group name	Staked	Cumulative Share	Score	Elected	
1-9		33,830,742	31.80%	100%	36 / 36	
Improve decentralization and network health by staking with a group below 4						
10	Figment Networks	3,185,723	34.80%	100%	2 / 2	Stake
11	Validator Capital	2,960,288	37.58%	100%	3 / 3	Stake
12	VibeStudio	2,892,858	40.30%	100%	5 / 5	Stake
13	Ledger by Figment	2,513,744	42.66%	100%	4 / 5	Stake
14	Censusworks	2,481,365	44.99%	100%	4 / 5	Stake
15	DSRV	2,407,327	47.26%	100%	2 / 2	Stake

Audit Scope

We at Hashlock audited the solidity code within the Celo project, the scope of work included a comprehensive review of the smart contracts listed below. We tested the smart contracts to check for their security and efficiency. These tests were undertaken primarily through manual line-by-line analysis and were supported by software-assisted testing.

Description	Celo Smart Contracts
Network	Ethereum
Language	Solidity
Audit Date	February, 2026
GitHub 1	https://github.com/ethereum-optimism/optimism/compare/op-deployer/v0.4.7...celo-org:optimism:celo-contracts/v4.1.0
Contract 1	ICeloSuperchainConfig.sol
Contract 2	IOptimismPortal2.sol
Contract 3	ISystemConfig.sol
Contract 4	IEthLiquidity.sol
Contract 5	IL1Block.sol
Contract 6	ISuperchainETHBridge.sol
Contract 7	L1CrossDomainMessenger.sol
Contract 8	L1StandardBridge.sol
Contract 9	OPContractsManager.sol
Contract 10	OptimismPortal2.sol
Contract 11	SystemConfig.sol
Contract 12	ETHLiquidity.sol
Contract 13	L1Block.sol
Contract 14	L2CrossDomainMessenger.sol

Contract 15	L2StandardBridge.sol
Contract 16	SuperchainETHBridge.sol
Contract 17	WETH.sol
Contract 18	CommonErrors.sol
Contract 19	CrossDomainMessenger.sol
Contract 20	OptimismMintableERC20.sol
Contract 21	StandardBridge.sol
GitHub 2	https://github.com/ethereum-optimism/optimism/compare/op-deployer/v0.5.2...celo-org:optimism:celo-contracts/v5.0.0
Contract 22	ICeloSuperchainConfig.sol
Contract 23	IOptimismPortal2.sol
Contract 24	ISystemConfig.sol
Contract 25	IETHLiquidity.sol
Contract 26	IL1Block.sol
Contract 27	ISuperchainETHBridge.sol
Contract 28	L1CrossDomainMessenger.so
Contract 29	L1StandardBridge.sol
Contract 30	OPContractsManager.sol
Contract 31	OptimismPortal2.sol
Contract 32	SystemConfig.sol
Contract 33	ETHLiquidity.sol
Contract 34	L1Block.sol
Contract 35	L2CrossDomainMessenger.sol
Contract 36	L2StandardBridge.sol
Contract 37	SuperchainETHBridge.so
Contract 38	WETH.sol

Contract 39	CommonErrors.sol
Contract 40	CrossDomainMessenger.sol
Contract 41	OptimismMintableERC20.sol
Contract 42	StandardBridge.sol
Audited GitHub Commit Hash 1	2c50c87bab8a37aeb363664b22ace82b898c2d8b
Audited GitHub Commit Hash 2	c138180e992f54502a62e3fcf803eb701e38b990

Security Rating

After Hashlock's Audit, we found the smart contracts to be **"Secure"**. The contracts all follow simple logic, with correct and detailed ordering. They use a series of interfaces, and the protocol uses a list of Open Zeppelin contracts.



Not Secure

Vulnerable

Secure

Hashlocked

The 'Hashlocked' rating is reserved for projects that ensure ongoing security via bug bounty programs or on chain monitoring technology.

All issues uncovered during automated and manual analysis were meticulously reviewed and applicable vulnerabilities are presented in the [Audit Findings](#) section. The list of audited assets is presented in the [Audit Scope](#) section and the project's contract functionality is presented in the [Intended Smart Contract Functions](#) section.

All vulnerabilities initially identified have now been resolved and acknowledged.

Hashlock found:

1 Medium severity vulnerabilities

4 QA

1 Gas Optimisations

Caution: Hashlock's audits do not guarantee a project's success or ethics, and are not liable or responsible for security. Always conduct independent research about any project before interacting.

Intended Smart Contract Functions

Claimed Behaviour	Actual Behaviour
ICeloSuperchainConfig.sol - Allows users to: - Library/Interface - callable helpers only - Allows admins to: - Library/Interface - callable helpers only	Contract achieves this functionality.
IOptimismPortal2.sol - Allows users to: - Library/Interface - callable helpers only - Allows admins to: - Library/Interface - callable helpers only	Contract achieves this functionality.
ISystemConfig.sol - Allows users to: - Library/Interface - callable helpers only - Allows admins to: - Library/Interface - callable helpers only	Contract achieves this functionality.
IETHLiquidity.sol - Allows users to: - Library/Interface - callable helpers only - Allows admins to: - Library/Interface - callable helpers only	Contract achieves this functionality.
IL1Block.sol - Allows users to: - Library/Interface - callable helpers only - Allows admins to: - Library/Interface - callable helpers only	Contract achieves this functionality.

ISuperchainETHBridge.sol - Allows users to: - Library/Interface - callable helpers only - Allows admins to: - Library/Interface - callable helpers only	Contract achieves this functionality.
L1CrossDomainMessenger.sol - Allows users to: - Send cross-chain messages to L2 contracts - Relay messages arriving from L2 messenger - Read current cross-domain message sender - Compute message nonce and base gas - Allows admins to: - Initialize with SystemConfig and OptimismPortal	Contract achieves this functionality.
L1StandardBridge.sol - Allows users to: - Deposit ETH to L2 via messenger - Deposit ERC20 to L2 via messenger - Deposit ETH or ERC20 to recipients - Receive finalized withdrawals from L2 bridge - Allows admins to: - Initialize bridge with messenger and SystemConfig	Contract achieves this functionality.
OPContractsManager.sol - Allows users to: - Deploy new OP Stack chains with roles - Validate deployed L1 contract	Contract achieves this functionality.

configurations - Compute batch inbox address from chain ID - Query blueprint and implementation addresses - Allows admins to: - Upgrade chains to latest implementations - Upgrade SuperchainConfig implementations - Add dispute game types and update prestates - Migrate chains to interop shared games	
OptimismPortal2.sol - Allows users to: - Deposit ETH and calldata to L2 - Deposit ERC20 as custom gas token - Prove L2 withdrawals against dispute games - Finalize proven withdrawals to L1 targets - Check withdrawal readiness and proof submitters - Allows admins to: - Initialize with SystemConfig and anchor registry - Set L2 gas paying token (SystemConfig only)	Contract achieves this functionality.
SystemConfig.sol - Allows users to: - Read system contract addresses and parameters - Check pause state and guardian address	Contract achieves this functionality.

- Query gas paying token metadata - Allows admins to: - Initialize config and set chain addresses - Update batcher, signer, gas limit, and scalars - Update EIP-1559 and operator fee parameters - Enable or disable feature flags - Transfer ownership and manage admin rights	
ETHLiquidity.sol - Allows users to: - Burn ETH liquidity (bridge-only caller) - Mint ETH liquidity (bridge-only caller) - Fund contract with ETH - Allows admins to: - None	Contract achieves this functionality.
L1Block.sol - Allows users to: - Read last known L1 block parameters - Read fee scalars and batcher hash - Query gas paying token metadata - Allows admins to: - Update L1 block values (depositor only) - Set gas paying token metadata (depositor only)	Contract achieves this functionality.
L2CrossDomainMessenger.sol - Allows users to: - Send cross-chain messages from L2 to L1 - Relay messages arriving from L1 messenger	Contract achieves this functionality.

- Read current cross-domain message sender - Allows admins to: - Initialize with L1 messenger address	
L2StandardBridge.sol - Allows users to: - Withdraw ETH to L1 via messenger - Withdraw ERC20 to L1 via messenger - Bridge ETH and ERC20 to L1 recipients - Receive finalized deposits from L1 bridge - Allows admins to: - Initialize bridge with other bridge address	Contract achieves this functionality.
SuperchainETHBridge.sol - Allows users to: - Send ETH to another superchain chain - Relay ETH from cross-chain message context - Allows admins to: - None	Contract achieves this functionality.
WETH.sol - Allows users to: - Query wrapped token name from gas token - Query wrapped token symbol from gas token - Allows admins to: - None	Contract achieves this functionality.
CommonErrors.sol - Allows users to:	Contract achieves this functionality.

- Library/Interface - callable helpers only - Allows admins to: - Library/Interface - callable helpers only	
CrossDomainMessenger.sol - Allows users to: - Send and relay cross-domain messages - Query sender and compute base gas - Allows admins to: - None	Contract achieves this functionality.
OptimismMintableERC20.sol - Allows users to: - Transfer and approve ERC20 balances - Sign permits for ERC20 approvals - Grant Permit2 max allowance behavior - Allows admins to: - Mint tokens to accounts (bridge only) - Burn tokens from accounts (bridge only)	Contract achieves this functionality.
StandardBridge.sol - Allows users to: - Bridge ETH/ERC20 to remote chain - Finalize inbound bridge messages - Allows admins to: - None	Contract achieves this functionality.

Code Quality

This audit scope involves the smart contracts of the Celo project, as outlined in the Audit Scope section. All contracts, libraries, and interfaces mostly follow standard best practices and to help avoid unnecessary complexity that increases the likelihood of exploitation, however, some refactoring were recommended to optimize security measures.

The code is very well commented on and closely follows best practice nat-spec styling. All comments are correctly aligned with code functionality.

Audit Resources

We were given the Celo project smart contract code in the form of GitHub access.

As mentioned above, code parts are well commented. The logic is straightforward, and therefore it is easy to quickly comprehend the programming flow as well as the complex code logic. The comments are helpful in providing an understanding of the protocol's overall architecture.

Dependencies

As per our observation, the libraries used in this smart contracts infrastructure are based on well-known industry standard open source projects.

Apart from libraries, its functions are used in external smart contract calls.

Severity Definitions

The severity levels assigned to findings represent a comprehensive evaluation of both their potential impact and the likelihood of occurrence within the system. These categorizations are established based on Hashlock's professional standards and expertise, incorporating both industry best practices and our discretion as security auditors. This ensures a tailored assessment that reflects the specific context and risk profile of each finding.

Significance	Description
High	High-severity vulnerabilities can result in loss of funds, asset loss, access denial, and other critical issues that will result in the direct loss of funds and control by the owners and community.
Medium	Medium-level difficulties should be solved before deployment, but won't result in loss of funds.
Low	Low-level vulnerabilities are areas that lack best practices that may cause small complications in the future.
Gas	Gas Optimisations, issues, and inefficiencies.
QA	Quality Assurance (QA) findings are informational and don't impact functionality. Supports clients improve the clarity, maintainability, or overall structure of the code.

Status Definitions

Each identified security finding is assigned a status that reflects its current stage of remediation or acknowledgment. The status provides clarity on the handling of the issue and ensures transparency in the auditing process. The statuses are as follows:

Significance	Description
Resolved	The identified vulnerability has been fully mitigated either through the implementation of the recommended solution proposed by Hashlock or through an alternative client-provided solution that demonstrably addresses the issue.
Acknowledged	The client has formally recognized the vulnerability but has chosen not to address it due to the high cost or complexity of remediation. This status is acceptable for medium and low-severity findings after internal review and agreement. However, all high-severity findings must be resolved without exception.
Unresolved	The finding remains neither remediated nor formally acknowledged by the client, leaving the vulnerability unaddressed.

Audit Findings

Medium

[M-01] OptimismPortal2#finalizeWithdrawalTransactionExternalProof - ETHLockbox Misaccounting With Custom Gas Token Withdrawals

Description

If `Features.ETH_LOCKBOX` is enabled while the chain uses a custom gas token, the portal can incorrectly treat `_tx.value` as both an ERC20 amount and an ETH amount. `finalizeWithdrawalTransactionExternalProof` unconditionally calls `ethLockbox.unlockETH(_tx.value)` before branching on the gas token, even though the custom gas token path interprets `_tx.value` as an ERC20 value.

This can move ETH out of the lockbox into `OptimismPortal2` during an otherwise successful custom-token withdrawal, leaving the ETH stuck in the portal and breaking lockbox accounting. The issue becomes reachable in migrations (ETH -> custom gas token) or via operational misconfiguration (enabling `ETH_LOCKBOX` on a custom gas token network).

Vulnerability Details

`OptimismPortal2` adds an ERC20-backed native asset flow by branching on `(address token,) = gasPayingToken()` and transferring ERC20 when `token != Constants.ETHER`.

However, the ETHLockbox flow remains unconditional and uses `_tx.value` to unlock/lock ETH regardless of whether `_tx.value` represents ETH or an ERC20-denominated native value.

In the custom gas token branch, `_tx.value` is explicitly used as an ERC20 transfer amount (`_balance -= _tx.value; IERC20(token).safeTransfer(...)`), which makes the earlier `unlockETH(_tx.value)` semantically incorrect.

On success, the unlocked ETH is never returned to the lockbox (the re-lock only happens on `!success`), so repeated withdrawals can drain ETH from the lockbox into the portal.

This creates a protocol-level fund loss/stuck-funds vector for any deployment where ETHLockbox is enabled while `systemConfig.gasPayingToken() != Constants.ETHER`.

The same semantic mismatch exists in the failure-path refund logic (`ethLockbox.lockETH{ value: _tx.value }()`), which also assumes `_tx.value` is ETH.

```
// src/L1/OptimismPortal2.sol
if (_isUsingLockbox()) {
    if (_tx.value > 0) ethLockbox.unlockETH(_tx.value);
}

(address token,) = gasPayingToken();
if (token == Constants.ETHER) {
    success = SafeCall.callWithMinGas(_tx.target, _tx.gasLimit, _tx.value, _tx.data);
} else {
    if (_tx.value != 0) {
        _balance -= _tx.value;
        IERC20(token).safeTransfer({ to: _tx.target, value: _tx.value });
    }
    success = _tx.data.length != 0 ? SafeCall.callWithMinGas(_tx.target, _tx.gasLimit, 0,
_tx.data) : true;
}
```

Proof of Concept

Deploy `OptimismPortal2` with `PROOF_MATURITY_DELAY_SECONDS = 0` and force-configure it to use a custom gas token address. Enable `Features.ETH_LOCKBOX` in the mocked `systemConfig` and set a funded `ethLockbox` address in storage.

Prepare a proven withdrawal entry so `checkWithdrawal` passes, then finalize a withdrawal with `_tx.value > 0`. Observe that the portal receives ETH from the lockbox even though the withdrawal is finalized via the ERC20 path, and that the ETH is not re-locked on success. This demonstrates lockbox ETH being drained into the portal by custom-token withdrawals.

```

contract
OptimismPortal2_FinalizeWithdrawalTransactionExternalProof_CustomGasToken_Lockbox_Test is
    OptimismPortal2_TestInit
{
    MockERC20 token;

    function setUp() public override {
        super.setUp();

        // TODO(opcm upgrades): remove skip once upgrade path is implemented
        skipIfForkTest("OptimismPortal2_Test: gas paying token functionality DNE on op
mainnet");

        token = new MockERC20("Test", "TST", 18);

        // Mock the gas paying token to be the ERC20 token for this suite.

        vm.mockCall(OptimismPortal2_FinalizeWithdrawalTransactionExternalProof_CustomGasToken_Loc
kbox_Test
            address(systemConfig),
            abi.encodeCall(systemConfig.gasPayingToken, ()),
            abi.encode(address(token), uint8(18))
        );
    }

    function
test_finalizeWithdrawalTransactionExternalProof_customGasToken_unlocksETHFromLockbox()
external {
        // Ensure the lockbox is enabled and wired.
        forceEnableLockbox(address(ethLockbox));
        assertTrue(isUsingLockbox());
        assertTrue(ethLockbox.authorizedPortals(optimismPortal2));

        // Make accounting deterministic: portal starts with 0 ETH, lockbox starts with
exactly the withdrawal value.
        vm.deal(address(optimismPortal2), 0);
        vm.deal(address(ethLockbox), _defaultTx.value);

        uint256 portalETHBefore = address(optimismPortal2).balance;

```

```

uint256 lockboxETHBefore = address(ethLockbox).balance;

// Seed the portal with custom gas token so the withdrawal can transfer the token
to the target.
token.mint(address(this), _defaultTx.value);
token.approve(address(optimismPortal2), _defaultTx.value);
optimismPortal2.depositERC20Transaction(
    address(bob), _defaultTx.value, 0, optimismPortal2.minimumGasLimit(0), false,
    ""
);
assertEq(optimismPortal2.balance(), _defaultTx.value);

// Prove and finalize the withdrawal with an external proof submitter (self).
optimismPortal2.proveWithdrawalTransaction({
    _tx: _defaultTx,
    _disputeGameIndex: _proposedGameIndex,
    _outputRootProof: _outputRootProof,
    _withdrawalProof: _withdrawalProof
});

game.resolveClaim(0, 0);
game.resolve();
vm.warp(block.timestamp + optimismPortal2.proofMaturityDelaySeconds() + 1
seconds);

optimismPortal2.finalizeWithdrawalTransactionExternalProof(_defaultTx,
address(this));

// Even though the gas paying token is an ERC20, the portal still unlocks ETH
from the ETHLockbox using
// `_tx.value`.
assertEq(address(optimismPortal2).balance, portalETHBefore + _defaultTx.value);
assertEq(address(ethLockbox).balance, lockboxETHBefore - _defaultTx.value);

// The withdrawal itself moves ERC20 value, not ETH.
assertEq(token.balanceOf(address(bob)), _defaultTx.value);
assertEq(optimismPortal2.balance(), 0);
}
}

```

Impact

If `ETH\_LOCKBOX` is enabled on a custom gas token deployment (or during an ETH -> custom gas token migration), any successful native-value withdrawal with `\_tx.value > 0` can unlock the same numeric amount of ETH from `ETHLockbox` into `OptimismPortal2` while also releasing the ERC20-backed native asset, potentially draining up to the entire lockbox ETH balance into a contract that is not intended to custody ETH and causing permanent stuck funds or lockbox insolvency without manual admin recovery.

Recommendation

Only execute ETHLockbox unlock/lock flows when the gas-paying token is ETH, and explicitly disallow enabling ETHLockbox when `isCustomGasToken()` is true.

At minimum, move the ETHLockbox unlock/lock logic inside the `token == Constants.ETHER` branch so `\_tx.value` cannot be interpreted as ETH when it actually represents ERC20-backed native value. Add an integration test that finalizes an ERC20-backed withdrawal with ETHLockbox enabled to ensure the contract fails fast (or never touches the lockbox) in this configuration.

```
diff --git a/packages/contracts-bedrock/src/L1/OptimismPortal2.sol
b/packages/contracts-bedrock/src/L1/OptimismPortal2.sol
@@
-      // If using ETHLockbox, unlock the ETH from the ETHLockbox.
-      if (_isUsingLockbox()) {
-          if (_tx.value > 0) ethLockbox.unlockETH(_tx.value);
-      }
+      (address token,) = gasPayingToken();
+      // ETHLockbox is only meaningful when the gas-paying token is ETH.
+      if (token == Constants.ETHER && _isUsingLockbox() && _tx.value > 0) {
+          ethLockbox.unlockETH(_tx.value);
+      }
@@
-      (address token,) = gasPayingToken();
+      if (token == Constants.ETHER) {
```

```
        success = SafeCall.callWithMinGas(_tx.target, _tx.gasLimit, _tx.value,
_tx.data);
    } else {
@@
-     if (_isUsingLockbox()) {
-         if (!success && _tx.value > 0) {
-             ethLockbox.lockETH{ value: _tx.value }();
-         }
-     }
+     if (token == Constants.ETHER && _isUsingLockbox() && !success && _tx.value > 0)
+     {
+         ethLockbox.lockETH{ value: _tx.value }();
+     }
```

Status

Resolved

QA

[Q-01] OptimismPortal2 - donateETH() traps ETH on Custom Gas Token chains

Description

`donateETH()` has no CGT guard. On CGT chains, ETH sent via this function is permanently trapped - invisible to `balance()` (returns `_balance`, not `address(this).balance`) and not recoverable through any withdrawal or admin path.

Recommendation

Add `if (token != Constants.ETHER) revert OnlyCustomGasToken()` or restrict to ETHLockbox only.

Status

Acknowledged (`donateETH` is not expected to provide a withdrawal path on CGT chains, and minimizing diff from the Optimism upstream is prioritized)

[Q-02] OptimismPortal2 - Missing isCustomGasToken() helper

Description

Portal inlines `(address token,) = gasPayingToken(); if (token == Constants.ETHER)` in 4 places with mixed comparison directions (`==` vs `!=`), while StandardBridge and CrossDomainMessenger use a centralized `isCustomGasToken()` helper. Increases maintenance risk.

Recommendation

Add `isCustomGasToken()` to OptimismPortal2 consistent with the other contracts.

Status

Acknowledged (Optional improvement; minimizing diff from Optimism is prioritized)

[Q-03] SystemConfig - Duplicate Import Of IOptimismPortal2

Description

`SystemConfig.sol` imports `IOptimismPortal2` twice, creating redundant declarations and unnecessary noise in the dependency graph.

This increases maintenance overhead and can confuse static analysis or automated diff review. While not a direct security issue, it is a clear correctness/quality regression in the updated code.

Recommendation

Remove the duplicated `import { IOptimismPortal2 } ...` line and keep a single import statement. This keeps the file clean and reduces the risk of future merge conflicts or inconsistent import ordering. Run the build/lint pipeline after the change to ensure no other import-ordering regressions are introduced.`

Status

Resolved

[Q-04] Contract - Misleading NotCustomGasToken Error Naming

Description

`NotCustomGasToken()` is reverted when the chain is using a custom gas token (e.g., `ETHLiquidty` and `SuperchainETHBridge`), even though the name reads as if the chain is not using one. This naming mismatch increases the chance of incorrect error handling and misunderstanding by integrators and auditors. It also makes it harder to reason about revert conditions when debugging production issues.

Recommendation

Rename the error to match its usage, e.g., `CustomGasTokenNotSupported()` or `OnlyEthGasToken()`, and update all call sites and interfaces. Alternatively, invert the error naming convention so the revert name describes the failure condition rather than the required mode. If backwards compatibility matters for consumers matching on selectors, introduce a new error and keep the old one as an alias during a deprecation window.

Status

Acknowledged (Original naming from Optimism; keeping smaller diff with upstream is preferred)

GAS

[G-01] Contract - Use Custom Errors For Custom Gas Token Checks

Description

Several newly added custom gas token checks use long revert strings (e.g., ``StandardBridge``, ``CrossDomainMessenger``, ``L2StandardBridge``), which increases deployment bytecode and runtime gas costs. These checks are hot-path validations for messaging and bridging and can be expressed more efficiently as custom errors. Inconsistent use of strings vs custom errors also reduces readability and makes error handling less uniform across the system.

Recommendation

Replace string-based ``require`` statements with custom errors (either newly defined or reusing ``CommonErrors.sol``), and standardize error names across contracts.

This reduces bytecode size and improves consistency for downstream tooling that matches on error selectors. Prefer reusing a single error (e.g., ``NotCustomGasToken()``) across ETH-only entrypoints to avoid fragmentation.

Status

Acknowledged (Original naming from Optimism; keeping smaller diff with upstream is preferred)

Centralisation

The Celo project values security and utility over decentralisation.

The owner executable functions within the protocol increase security and functionality but depend highly on internal team responsibility.



Centralised

Decentralised

Conclusion

After Hashlock's analysis, the Celo project seems to have a sound and well-tested code base, now that our vulnerability findings have been resolved and acknowledged. Overall, most of the code is correctly ordered and follows industry best practices. The code is well commented on as well. To the best of our ability, Hashlock is not able to identify any further vulnerabilities.

Our Methodology

Hashlock strives to maintain a transparent working process and to make our audits a collaborative effort. The objective of our security audits is to improve the quality of systems and upcoming projects we review and to aim for sufficient remediation to help protect users and project leaders. Below is the methodology we use in our security audit process.

Manual Code Review:

In manually analysing all of the code, we seek to find any potential issues with code logic, error handling, protocol and header parsing, cryptographic errors, and random number generators. We also watch for areas where more defensive programming could reduce the risk of future mistakes and speed up future audits. Although our primary focus is on the in-scope code, we examine dependency code and behaviour when it is relevant to a particular line of investigation.

Vulnerability Analysis:

Our methodologies include manual code analysis, user interface interaction, and white box penetration testing. We consider the project's website, specifications, and whitepaper (if available) to attain a high-level understanding of what functionality the smart contract under review contains. We then communicate with the developers and founders to gain insight into their vision for the project. We install and deploy the relevant software, exploring the user interactions and roles. While we do this, we brainstorm threat models and attack surfaces. We read design documentation, review other audit results, search for similar projects, examine source code dependencies, skim open issue tickets, and generally investigate details other than the implementation.

Documenting Results:

We undergo a robust, transparent process for analysing potential security vulnerabilities and seeing them through to successful remediation. When a potential issue is discovered, we immediately create an issue entry for it in this document, even though we have not yet verified the feasibility and impact of the issue. This process is vast because we document our suspicions early even if they are later shown to not represent exploitable vulnerabilities. We generally follow a process of first documenting the suspicion with unresolved questions, and then confirming the issue through code analysis, live experimentation, or automated tests. Code analysis is the most tentative, and we strive to provide test code, log captures, or screenshots demonstrating our confirmation. After this, we analyse the feasibility of an attack in a live system.

Suggested Solutions:

We search for immediate mitigations that live deployments can take and finally, we suggest the requirements for remediation engineering for future releases. The mitigation and remediation recommendations should be scrutinised by the developers and deployment engineers, and successful mitigation and remediation is an ongoing collaborative process after we deliver our report, and before the contract details are made public.

Disclaimers

Hashlock's Disclaimer

Hashlock's team has analysed these smart contracts in accordance with the best industry practices at the date of this report, in relation to: cybersecurity vulnerabilities and issues in the smart contract source code, the details of which are disclosed in this report, (Source Code); the Source Code compilation, deployment, and functionality (performing the intended functions).

Due to the fact that the total number of test cases is unlimited, the audit makes no statements or warranties on the security of the code. It also cannot be considered as a sufficient assessment regarding the utility and safety of the code, bug-free status, or any other statements of the contract. While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only. We also suggest conducting a bug bounty program to confirm the high level of security of this smart contract.

Hashlock is not responsible for the safety of any funds and is not in any way liable for the security of the project.

Technical Disclaimer

Smart contracts are deployed and executed on a blockchain platform. The platform, its programming language, and other software related to the smart contract can have their own vulnerabilities that can lead to attacks. Thus, the audit can't guarantee the explicit security of the audited smart contracts.

About Hashlock

Hashlock is an Australian-based company aiming to help facilitate the successful widespread adoption of distributed ledger technology. Our key services all have a focus on security, as well as projects that focus on streamlined adoption in the business sector.

Hashlock is excited to continue to grow its partnerships with developers and other web3-oriented companies to collaborate on secure innovation, helping businesses and decentralised entities alike.

Website: hashlock.com.au

Contact: info@hashlock.com.au



#hashlock.